



PROFIA
escola de programação

DELIVERY RUSH

GAMEMAKER





Delivery Rush: O cérebro do personagem

Hoje vamos configurar as regras gerais do jogo e programar o comportamento inteligente do nosso personagem. Inicialmente, precisamos criar **variáveis globais de controle no Script**

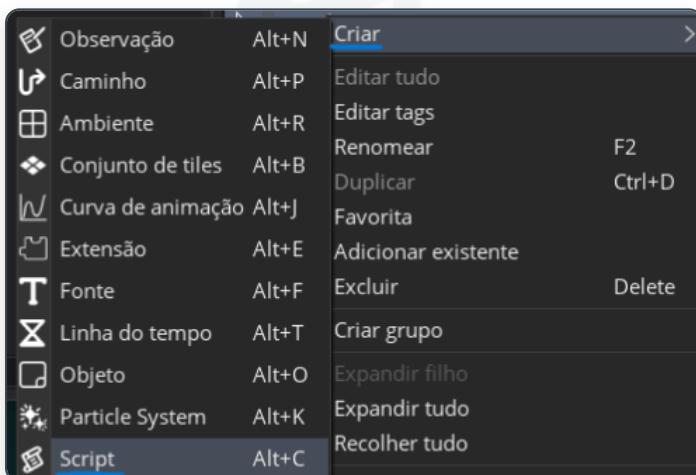
1. O Poder dos Scripts

O QUE É UM SCRIPT?

Pense no Script como um "Manual de Regras" do jogo. É um local para escrever códigos que serão utilizadas no jogo inteiro ou quando não podemos atribuir esse código a um objeto.

Criando Variáveis Globais

Algumas informações precisam existir mesmo se o personagem for destruído ou se mudarmos de fase. Para isso, usamos `global`. antes do nome. Criando uma variável Global.



SCRIPT: INICIALIZAR_JOGO

```
global.dinheiro = 0;  
global.tem_moto = false;  
  
global.valor_entrega = 5;  
global.tempo_espera = 60;  
global.energia = 100;
```

💡 Por que Global?

Se o dinheiro fosse uma variável local do jogador, quando ele saísse da tela ou o jogo reiniciasse, o dinheiro voltaria a zero. Com `global`, o saldo fica salvo!



2. Iniciando a Movimentação

Agora que temos as regras globais, vamos para o cérebro do jogador (evento **Step**). A primeira coisa que ele precisa fazer é calcular o quão rápido ele pode correr.

OBJ_PLAYER > EVENTO: STEP

```
var velocidade_base = 15;  
if (global.tem_moto) velocidade_base = 30;
```

VARIÁVEIS TEMPORÁRIAS (VAR)

Repare que usamos **var** antes de **velocidade_base**. Isso cria uma **Variável Temporária**.

Ela nasce, é usada nesse cálculo e morre imediatamente quando o código termina. Isso é ótimo para economizar a memória do computador, já que não precisamos guardar esse valor para sempre!

3. Controlando os Estados

Para controlar se o jogador está "PARADO", "INDO" ou "VOLTANDO", poderíamos usar vários **if** e **else if**, mas isso deixaria o código uma bagunça.

Switch Case: A Escolha Profissional

Em vez de perguntar "É A? Não. É B? Não. É C?", o **Switch Case** funciona como uma central de distribuição que manda o código direto para o lugar certo.

JEITO "BAGUNÇADO" (IF)

```
if (estado == "A") {...}  
else if (estado == "B") {...}  
else if (estado == "C") {...}
```

JEITO LIMPO (SWITCH)

```
switch (estado) {  
  case "A": ... break;  
  case "B": ... break;  
}
```

💡 Por que Switch?

Além de ser mais organizado visualmente, o Switch é processado mais rápido pelo computador quando temos muitas opções de estados.



4. Detalhando a Lógica

Agora vamos preencher o nosso Switch com a lógica de movimento do entregador.

CASE "PARADO"

Se ele está parado, podemos clicar nele para iniciar uma entrega.

```
case "PARADO":  
    pode_clicar = true;  
    break;
```

CASE "INDO"

Aumenta o X para andar. Se sair da tela, teleporta para o início.

```
case "INDO":  
    x += velocidade_movimento;  
    // Se saiu da tela pela direita  
    if (bbox_left > room_width) {  
        x = 0 - sprite_width; // Teleporta p/ esquerda  
        estado = "VOLTANDO";  
    }  
    break;
```

CASE "VOLTANDO"

Continua andando até chegar na loja e receber o pagamento.

```
case "VOLTANDO":  
    x += velocidade_movimento;  
    // Se chegou na loja (posição inicial)  
    if (x >= posicao_inicial_x) {  
        x = posicao_inicial_x; // Trava na posição  
        estado = "PARADO";  
        global.dinheiro += global.valor_entrega;  
    }  
    break;
```

Desafio Visual: Feedback de Pagamento

O jogador ganhou dinheiro, mas nada aparece na tela. Vamos dar um feedback visual!

Missão: Crie um objeto chamado **obj_nota** e faça com que sempre que o jogador receber por uma entrega, uma nota saia dele em direção a lanchonete.

Dicas: Lembre-se do `Instance_Create_Layer()` e de que a nota precisa de uma **vspeed** para se mover.



Anotações:

/ 10

/ 20